

Razorpay Setup and Debug Guide (Flutter)

Date: May 18, 2026

Project: razorpaydemo

1. Summary

The payment flow is now working end-to-end, and most runtime issues were caused by setup/configuration, not Flutter syntax errors.

Main blockers we resolved:

- Missing or invalid Razorpay Key ID in app runtime.
- Checkout opening with no available payment methods for account mode.
- Success page crashing due to an invalid Lottie animation package.
- Payment options hardcoded in service layer with less-safe structure.

2. Razorpay Account Setup (Correct Sequence)

1. Create account at <https://dashboard.razorpay.com/signup>.
2. Log in and switch to Test Mode (top-right toggle).
3. Open Account & Settings -> API Keys.
4. Generate keys and copy only Key ID (starts with rzp_test_... in test mode).
5. Keep Key Secret private (backend/server only, never in Flutter UI code).
6. Open Account & Settings -> Payment Methods and enable methods you want to test.
7. Run payment once with test key and test methods.
8. For production, complete KYC/onboarding and use rzp_live_... Key ID.

3. Production-Safe Payment Flow

1. App requests your backend to create order.
2. Backend calls Razorpay Orders API with Key ID + Key Secret.
3. Backend returns order_id to app.
4. App opens checkout with key + order_id + amount.
5. On success callback, app sends payment_id/order_id/signature to backend.
6. Backend verifies signature and marks booking successful.
7. Fulfill service only after verified success.

4. Issues Found and Root Cause

Issue A: "Razorpay key is missing/invalid"

Root cause:

- App was running without --dart-define key in some runs.

Fix applied:

- Added fallback + one-time local key save + runtime validation.
- File: lib/controller/payment_controller.dart.

Issue B: "No appropriate payment method found"

Root cause:

- Key mode/payment-method configuration mismatch in Razorpay dashboard.

Fix applied:

- Added explicit failure guidance when this message appears.
- File: lib/controller/payment_controller.dart.

Issue C: Red error on Success screen (Lottie assertion)

Root cause:

- DotLottie file caused parser assertion: startFrame == endFrame.

Fix applied:

- Switched to stable JSON animation asset.
- Added errorBuilder fallback icon so success screen never crashes.
- Files: lib/views/success_screen.dart, assets/animations/payment_success.json.

Issue D: Weak payment option wiring

Root cause:

- Payment options were assembled in service with placeholder key and static values.

Fix applied:

- Options now built in controller with dynamic amount, retry, timeout, and validated key.
- File: lib/controller/payment_controller.dart.

Issue E: Platform/network preconditions not explicit

Root cause:

- No early checks for unsupported platforms and poor recovery message.

Fix applied:

- Added platform guard and checkout safety timer with actionable message.
- File: lib/controller/payment_controller.dart.

5. Code Improvements Made

5.1 Controller hardening

File: lib/controller/payment_controller.dart

- Added key validation for rzp_test_/rzp_live_.
- Added optional orderId parameter support.
- Added loading-state guard against duplicate taps.
- Added smarter error mapping and user-safe snackbars.
- Added local key persistence using shared_preferences.

5.2 Service cleanup

File: lib/services/razorpay_service.dart

- Removed hardcoded options from service.
- Service now accepts options from controller.
- Kept service focused on SDK open/close lifecycle.

5.3 Ticket screen integration

File: lib/views/ticket_screens.dart

- Passes movie price and movie name into payment call.
- Uses amount conversion from rupees to subunits in controller.

5.4 Success screen resilience

File: lib/views/success_screen.dart

- Uses assets/animations/payment_success.json.
- Added errorBuilder fallback icon if animation parsing fails.

5.5 Platform config updates

Files:

- android/app/src/main/AndroidManifest.xml
- ios/Runner/Info.plist

Changes:

- Added Android INTERNET permission.
- Added iOS UPI app query schemes for better app discovery.

5.6 Dependency update

File: pubspec.yaml

- Added shared_preferences for one-time key storage.

6. Security Mistakes to Avoid

1. Never put Key Secret in Flutter app.
2. Never push Key Secret to git.
3. Rotate compromised Key Secret immediately in Razorpay dashboard.
4. Prefer passing Key ID via --dart-define or secure runtime config.
5. Use backend order creation + signature verification for real bookings.

7. Commands for Smooth Local Run

Option A (recommended): pass key via dart-define

```
flutter run --dart-define=RAZORPAY_KEY=rzp_test_your_key_id
```

Option B: use in-app one-time key prompt

- Tap Book Ticket.
- Enter Key ID once.
- App stores key locally and reuses it.

Full refresh if stale state exists

```
flutter clean
```

```
flutter pub get
```

```
flutter run
```

8. Interpreting Common Logs

- Chromium frame-latency warnings: usually noisy/non-blocking in checkout webview.
- Skipped frames: often performance noise, not payment API failure.
- Real failure is usually in Razorpay callback message or dashboard config.

9. Verification Completed in This Project

- flutter analyze: passed.
- flutter test: passed.
- Payment success screen no longer crashes due to animation parser issue.

10. Final Go-Live Checklist

1. Use rzp_live_ Key ID only in release build.
2. Complete backend Orders API and signature verification.

3. Enable required payment methods in live mode.
4. Configure webhooks for payment.captured, payment.failed, refund events.
5. Monitor and log callback payloads and verification results.

11. Official References

1. Flutter Integration Steps:

<https://razorpay.com/docs/payments/payment-gateway/flutter-integration/standard/integration-steps/>

2. Build Integration (Flutter):

<https://razorpay.com/docs/payments/payment-gateway/flutter-integration/standard/build-integration/>

3. API Keys on Dashboard:

<https://razorpay.com/docs/payments/dashboard/account-settings/api-keys/>

4. Test and Live Modes:

<https://razorpay.com/docs/payments/dashboard/test-live-modes/>